Kursal: A peer to peer encrypted messenger

Erik LP.
Kodeur_Kubik
kubik@kursal.chat

Armand H.
Arlo
arlo@kursal.chat

Abstract

Kursal proposes an end-to-end encrypted, peer-to-peer protocol for exchanging messages and files over a decentralized network. It uses Signal's encryption methods combined with libp2p's protocols to achieve a fully decentralized setup. The network relies on volunteer nodes: the more nodes that are connected, the more robust the network gets. Any node can offer itself as a public relay to help others connect across NATs or behind firewalls. Kursal's objective is to combine security, decentralization and usability into one unified application.

Version: 1.3
Date: June 27, 2026
Website: https://kursal.chat/

1. Introduction

Messaging platforms have become the most widely adopted systems for online communication, file transfers and calls. However, commonly used messengers either do not encrypt messages on their servers or lack transparency. Others compromise user privacy by collecting private user data. Even transparent systems are vulnerable. Governments could still access centralized hosting servers, block or filter content, censor users, or spy on them. This is where Kursal steps in as a potential solution: all messages, files, and calls are end-to-end encrypted and transit over a decentralized network hosted by volunteers.

1.1. Motivation

Growing centralization in digital communication threatens user privacy and autonomy. Proposals like the European Union's Chat Control initiative to scan user messages for harmful content highlight the risk of surveillance becoming normalized under the guise of safety. Kursal aims to demonstrate a different path: a fully decentralized, peer-to-peer protocol for private communication based on modern cryptography. We believe privacy and security can coexist through open, verifiable, and community-driven technology. We do not endorse crimes but believe in non-privacy-intrusive ways of countering them. Our goal is to demonstrate that decentralized peer-to-peer encrypted messaging can achieve the same level of usability and reliability as centralized platforms.

2. First Contact

There are three ways for two peers "Alice" and "Bob" to initiate communication over the Kursal network. Each method has its advantages and disadvantages:

- **One-Time Password**: Easy to share. Takes time to publish on the network, is single-use and requires computation.
- **Long-Term Code**: Multiple usage code that requires no computation. File size of 6 KB that must be transferred via an external channel.
- **Nearby Share**: Easy to share. Devices must be on the same Wi-Fi network (which must support mDNS) to initiate the connection.

2.1. With a One-Time Password

Alice generates a one-time password (referred to as OTP) composed of 8 random words. The interface can also display this code as a QR-code that can be scanned by the other user. The word dataset mainly comes from eff.org/dice and we filtered and added words, resulting in a total of around 11 080 words. Alice computes the argon2id [1] hash of this OTP, referred to as `OTP_H`; for benchmarks and the exact hashing configuration, see Section 8.2. This hash becomes the key that encrypts an initial payload `(PQXDH_payload, Peer ID, pubkey, relays)` with XChaCha20-Poly1305 [2], where `PQXDH_payload` is a PQXDH [3] pre-key bundle. Alice then publishes the encrypted payload to the distributed hash table (DHT) [4] under the

key `SHA-256(OTP_H)` (see Section 4 for the record format). The DHT key is public, so it is derived by hashing `OTP_H` rather than using it directly: a relay can see where the record is stored but cannot derive the key that decrypts it. The DHT entry expires and is automatically removed after 10 minutes. The public sign key (`pubkey`) is a public Dilithium-5 [5] key that will later prove the integrity of messages. The `relays` entry contains addresses on which Alice can be contacted (direct and relays). Bob may now fetch the corresponding DHT entry by first hashing the OTP he received from an external source. He can then complete the PQXDH protocol and initiate a Double Ratchet [6] extended by the ML-KEM Braid Protocol [7]. For Alice to also complete the PQXDH protocol, Bob must send the final PQXDH payload, his public signing key, his public listening addresses (like Alice's `relays`) and a Double Ratchet initial message through a relay.

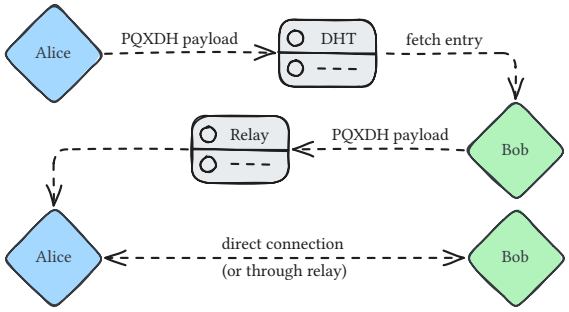

Figure 1: Direct contact with an OTP

2.2. With a Long-Term Code

Exchanging individual one-time passwords can be time-consuming. To simplify long-term contact management, Alice creates an object containing her peer ID, an initial PQXDH payload [3], an ephemeral public key, a public Dilithium-5 [5] signing key as well as public addresses to reach Alice (direct IP or via relays). The PQXDH payload does not contain a one-time pre-key as this payload can be used multiple times by different users. The ephemeral key is kept in cache by Alice and deleted once the Long-Term Code (referred to as LTC) is renewed or invalidated. This object can then be encoded into a ".kursal" file (which is approximately 6 KB in size). After receiving the file, Bob decodes this object and completes the PQXDH protocol by sending the PQXDH payload, his public signing key and his public listening addresses to Alice. They can initiate a Double Ratchet [6] extended by the ML-KEM Braid Protocol [7] as described in Section 2.1. Because one bundle is used by many contacts, its pre-keys are not single-use: there is no one-time pre-key, and the Kyber pre-key is kept instead of being deleted after use. This slightly weakens forward secrecy for the first messages of a conversation: until the Double Ratchet takes its first step, those messages rely only on long-lived keys, so they could be exposed if the device is later compromised and the early traffic was recorded. Every later message, and every contact added through a One-Time Password or Nearby Share, keeps full forward secrecy.

2.3. Nearby Share

The Nearby Share process allows devices to discover each other over the same local network via multicast DNS (mDNS) or via Bluetooth (BLE). To protect user privacy and prevent unsolicited key exchanges, this is a two-step process:

First, the discovery phase: devices broadcast a beacon containing a randomly generated, ephemeral "adjective-animal" username (e.g., "Orange Mouse"). Users can identify each other using these temporary names.

Second, the handshake phase: one user initiates a connection, and the other must explicitly accept the request. After mutual consent, the public keys and initial PQXDH payloads [3] are exchanged. Both mDNS and Bluetooth are implemented in Kursal.

2.4. Security Code

Each security code is unique to each contact and should be verified as soon as the communication is established. It should match on both sides and if not, it very likely means a man-in-the-middle attack is happening. Security codes should **not** be verified by sending them over the chat because in the case of a man-in-the-middle attack, the attacker could impersonate this code as well.

It is computed deterministically from both parties' identity and signing keys, so each side obtains the same result. The two public identity keys and the two Dilithium-5 public signing keys are sorted and concatenated, then hashed with

SHA-256. The digest is turned into 8 groups of 4 decimal digits and shown in the interface.

3. Continuous Communication

Kursal is based on rust libp2p's swarm architecture. It supports both TCP and QUIC protocols and automatically finds the optimal networking solution. Each connection initially starts from a relay and will, if possible, be upgraded using DCUTR (Direct Connection Upgrade Through Relay). This upgrade can be a direct peer-to-peer connection or both peers can attempt a NAT traversal using hole punching. If both of those fail, the connection is maintained through the relay. Each node is identified by its peer ID, which rotates regularly to prevent tracking.

Each message receives a delivery receipt in reply. The receipt travels back through the same end-to-end encrypted session, so its arrival already proves it came from the recipient and no separate signature is needed. A message is marked as delivered only once this receipt arrives.

Figure 2: Delivery receipt for a received message

3.1. Peer Rediscovery

Because peer IDs rotate regularly, a contact's transport address can change while one of the two parties is offline. When a node rotates its identity, or its reachable addresses otherwise change, it sends an address-update message over its existing authenticated, encrypted session to every reachable contact, and repeats this announcement periodically as a heartbeat. Contacts that are offline at that moment instead recover the new routing through the offline mailbox described in Section 5: every stored bundle carries the sender's current peer ID and addresses, so retrieving queued messages is by itself enough to learn how to reach the sender again.

4. Distributed Hash Table

Several Kursal mechanisms publish records to a Kademlia distributed hash table [4] hosted by the volunteer network, including one-time password exchange (Section 2.1) and offline messaging (Section 5). Because any node may write to the DHT, every record uses a uniform, self-policing format that storing nodes can validate before accepting it.

A record carries its key, its value, a publication timestamp, and a proof-of-work nonce. A storing node checks only the payload size, the timestamp (within the record's lifetime and at most two minutes in the future), and the proof of work. It does not verify authenticity, and it does not need to. A record's value is always encrypted under a key that only the legitimate parties know: the one-time password for an OTP record, or the shared mailbox root for a bundle. A forged record simply fails to decrypt, so no signature is required.

The proof of work requires the publisher to find a nonce such that the SHA-256 hash of the record (its key, value and timestamp) together with that nonce falls below a fixed target. Two targets exist: a lighter one for short-lived records and a heavier one for long-lived records such as offline bundles. Every write therefore costs measurable computation, which makes flooding the DHT, or a single mailbox, expensive, while verification stays a single hash. Records expire automatically: short-lived records live for 10 minutes, and long-lived records for up to three weeks, after which the network discards them.

5. Offline Messaging

Two contacts are often offline at the same time, so Kursal stores messages for absent peers in the distributed hash table (Section 4) instead of relying on a server. Each pair of contacts shares a private, append-only mailbox that only the two of them can locate or read.

5.1. Mailbox Keys

The two peers build the mailbox roots from two shared secrets: a classical one and a post-quantum one. The classical secret is a Curve25519 Diffie-Hellman agreement between their identity keys. The post-quantum secret comes from ML-KEM: during first contact one peer encapsulates to the other's Kyber pre-key and sends the resulting ciphertext, which the other decapsulates, so both end up with the same value. The two

secrets are combined through HKDF-SHA256, using both identity public keys as the salt and a different label for each direction, to produce two 32-byte root keys, one per direction. An attacker would have to break both Curve25519 and ML-KEM to recover them, so the mailbox is as quantum-safe as the rest of Kursal. And because only the two peers can compute the roots, no one else can even find the mailbox.

Each direction keeps a counter that only increases. The key under which bundle number n is stored, which we call its tag, is derived from the root and the counter with HKDF-SHA256. Successive tags are therefore pseudo-random and unlinkable: an observer cannot tell that two records belong to the same conversation, nor count how many messages a pair exchanges.

5.2. Bundling and Publication

Outgoing messages, already encrypted with the Double Ratchet [6], are queued and grouped into a bundle. A bundle is flushed as soon as any of three thresholds is met: fifteen queued messages, thirty seconds elapsed since the first message was queued, or 50 KB of accumulated ciphertext. Batching amortizes the per-record proof-of-work cost over several messages.

The bundle contains the queued ciphertexts together with the sender's current peer ID and reachable addresses. It is encrypted with XChaCha20-Poly1305 [2] under a per-bundle key derived from the root and the counter, then published as a long-lived, proof-of-work record (Section 4). The sender's public signing key is deliberately omitted from the record, so a stored bundle reveals nothing about who published it. A bundle is re-published until it is retrieved or until it expires after three weeks.

5.3. Retrieval

A returning peer looks for new bundles by computing the next tags from its receiving root and fetching a window of sixty-four counters in parallel. For each bundle found, the peer decrypts it. Decryption only succeeds with the shared mailbox key, so a bundle that decrypts is necessarily from the contact. The peer then advances its counter and applies the messages it contains. It also adopts the embedded peer ID and addresses, reconnecting directly to a contact whose transport identity rotated while it was away (Section 3.1). Once back online, the peer confirms each retrieved message with the delivery receipt described in Section 3. If a counter stays empty, for example because a bundle has not yet propagated, the peer retries; after 48 hours without progress it skips the gap so that a single lost bundle cannot stall the conversation permanently.

6. Data Streams

In order to transfer a large amount or a continuous stream of data like file transfers and calls, Kursal cannot encrypt each packet with Double Ratchet as this method would be too computationally intensive. Therefore, each time a data stream is about to happen (calls, file transfers, …), both agree on a random, temporary key to encrypt all following packets using the XChaCha20-Poly1305 [2] protocol. This allows lightweight encryption for one-time transfers. Each peer generates a random 32-byte key that is sent to the other via Double Ratchet. Both peers derive a shared key by passing the concatenation of their two keys through HKDF-SHA256. This ensures a cryptographically secure mix that cannot be guessed by compromising one of the original keys. XChaCha20-Poly1305 uses a large random nonce, so reuse is not a concern even for long transfers.

7. Implementation

We propose a Kursal implementation written in Rust available at https://github.com/KursalChat/Kursal under the GNU AGPLv3 license, available on Linux, macOS and Windows. Our implementation uses Signal's libsignal and libp2p's rust-libp2p libraries.

We plan to improve this initial version over time by adding group conversations and broadcasting channels. These features are listed on our TODO list and will be implemented over time.

8. Risk Mitigation

8.1. Operating System

If the operating system itself is compromised, Kursal cannot guarantee the integrity of messages. All encryption keys, which are stored in the keychain of the computer could be accessed by malware or by the operating system itself. It is up to the user to choose a trusted operating system and keep it secure.

8.2. DHT Brute Forcing

To evaluate possible risks of pre-calculating every possible OTP and its corresponding hash, we can estimate how long it would take to compute. Given a computer hashing speed of $\mathrm{CS} = 1000$ hashes per second and per device, a time period of 1000 years, which approximates to $T \approx 3 * 10^{10}$ seconds. Rounding the wordlist down to 11 000 words, the number of possible OTPs is $P = 11000^8 = 2.14 * 10^{32}$. We can approximate the number of devices D needed with this hashing speed to calculate every possibility:

$$D = \frac{P}{\mathrm{CS} * T} \approx 7.1 * 10^{18} \text{ devices}$$

This means that in order to intercept 0.1% of the DHT entries, one would need to compute at 1000 hashes per second for 1000 years with $7.1 * 10^{15}$ devices. And for one interception in a million, it drops to $7.1 * 10^{12}$ devices for the next 1000 years at 1000 hashes per second. For more details about what DHT interceptions lead to, refer to Section 8.3.

The Argon2id configuration used is configured to take a significant amount of time on modern computers. We are therefore well under the theoretical 1000 hashes per second, making brute-forcing impractical and future-proof. Here are the results of the hashing benchmark (Kursal uses those exact settings) with: $\mathrm{memory} = 256\ \mathrm{MiB}$, $\mathrm{iterations} = 2$, $\mathrm{parallelism} = 1$. The high memory limit prevents massive parallelization.

CPU	Time per iteration	Iter. per sec.
Intel Core i5-14600kf	709.5 ms 35.6 ms (threaded)	28.07
MacBook Pro M5	376.2 ms 37.7 ms (threaded)	26.53
Intel Core i5-12400F	629.3ms 52.6ms (threaded)	19.01
Intel Core i5-10310U	2234ms 280ms (threaded)	3.57

Table 1: Hashing Benchmark: 1000 iterations

The assumed value of $\mathrm{CS} = 1000$ hashes per second is an extremely high speed that is not reached by modern computers. It is therefore theoretically possible to brute force the DHT, but very unlikely. If such an attack ever became feasible, the fixed Argon2id salt could be rotated to a new value, forcing attackers to recompute every possibility from scratch.

8.3. DHT Interception

Intercepting the DHT is defined by accessing an encrypted initial PQXDH payload by finding the OTP. In a hypothetical scenario, a hacker could pre-compute all pairs of OTP and their hash and decrypt the payload. The initial payload does not contain any sensitive information as it is only made up of public keys. The hacker could attempt a man-in-the-middle attack but it could be detected by a different security code (see Section 2.4).

8.4. Malicious Relays

Relays, even malicious, will never be able to read your messages as they are encrypted. However, they can analyze your messaging patterns if those patterns are left unprotected. This is why peer IDs rotate regularly to prevent tracking your communication patterns. Additionally, relays have access to your IP address if no proxy or VPN is used.

8.5. DHT Flooding

Because any node may publish to the distributed hash table, a malicious actor could try to exhaust storage or bury a victim's mailbox under spurious records. Every write is gated by a proof of work (Section 4). Flooding the network, or even

a single mailbox tag, then costs the attacker real computation for every record, while an honest node still checks each one with a single hash. Long-lived records such as offline bundles carry the heavier difficulty.

8.6. Offline Metadata

Offline bundles are stored on untrusted volunteer nodes. Because each mailbox tag is derived from a secret only the two contacts share, a storing node sees only an opaque, random-looking key: it cannot tell which user a record belongs to, tell that two records belong to the same conversation, or count how many messages a pair exchanges. Because the mailbox roots are post-quantum (Section 5), this stays true even against a future quantum attacker, which cannot work out the tags from the public identity keys alone. A bundle carries no signature and no sender key of any kind, so a stored bundle cannot be tied to whoever published it. The messages inside are Double Ratchet [6] ciphertext, so their confidentiality and forward secrecy come from the ratchet itself.

Bibliography

[1] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: the memory-hard function for password hashing and other applications," technical report, Mar. 2017. [Online]. Available: https://github.com/P-H-C/phc-winner-argon2/blob/master/argon2-specs.pdf

[2] Y. Nir and A. Langley, "ChaCha20 and Poly1305 for IETF Protocols," technical report RFC 8439, May 2026. [Online]. Available: https://datatracker.ietf.org/doc/rfc8439/

[3] E. Kret and R. Schmidt, "The PQXDH Key Agreement Protocol," technical report, May 2023. [Online]. Available: https://signal.org/docs/specifications/pqxdh/

[4] Raulk, J. Hiesey, and M. Inden, "libp2p Kademlia DHT Specification," technical report, Dec. 2022. [Online]. Available: https://github.com/libp2p/specs/blob/master/kad-dht/README.md

[5] L. Ducas *et al.*, "CRYSTALS-Dilithium: A Lattice-Based Digital Signature Scheme," technical report, Feb. 2021. [Online]. Available: https://eprint.iacr.org/2017/633.pdf

[6] T. Perrin, M. Marlinspike, and R. Schmidt, "The Double Ratchet Algorithm," technical report, Sep. 2025. [Online]. Available: https://signal.org/docs/specifications/doubleratchet/

[7] R. Schmidt, "The ML-KEM Braid Protocol," technical report, Feb. 2025. [Online]. Available: https://signal.org/docs/specifications/mlkembraid/